

# MACHINE LEARNING IN FINANCE

## MODULE 7 | LESSON 1

---

# MODEL SELECTION AND HYPERPARAMETER TUNING

---

|                        |                                                              |
|------------------------|--------------------------------------------------------------|
| <b>Reading Time</b>    | 50 minutes                                                   |
| <b>Prior Knowledge</b> | Machine learning                                             |
| <b>Keywords</b>        | Hyperparameter, grid search, random search, Cross validation |

---

*In the previous modules, we have trained and evaluated the performance of machine learning models. Machine learning models have a number of parameters that the data learn; model parameters are found by fitting our model on the training dataset. Hyperparameters on the other hand cannot be learned by our normal training process and are fixed before we begin the training process. In this module, we will learn more about hyperparameters. In this lesson, we will begin by discussing how to select the best performing model and the various hyperparameter tuning strategies.*

## 1. Improving Model Performance

Before putting our machine learning model(s) to production, we should make sure that the model gives the best possible solution devoid of bias and can generalize well on new unseen data.

We will cover the following topics in this section.

- Overfitting and underfitting.
- Train, validation, and test sets.
- Cross-validation.

We begin with data splitting into train, test, and validation sets.

### 1.1 Train, validation and tests sets

It is advisable to split the data into training, testing, and validation sets especially if we want to tune our hyperparameters. The validation set helps us get an unbiased evaluation of our model performance and can therefore tune our model hyperparameters before passing the model to the test dataset.

Given a dataset, we can get the train, validation, and test set by using one of the following:

- Random Split
- Stratified Split

Random split involves splitting the original data into pre-defined proportions representing the train, validation, and test sets by first shuffling the data so as to overcome the challenge of there existing a pattern based on the data indices.

Stratified split splits the train, validation, and test sets according to the proportion of samples in the target variable. Stratified split is often used when our data is imbalanced.

Let us see an example of this using the California housing dataset whose target variable is the house price median.

```
In [ ]: from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X, y = housing.data, housing.target * 10
```

```
In [ ]: X.head()
```

```
In [ ]: y.head()
```

We will use the decision tree regressor to predict the median house price with an interest in getting the generalization of our model. So, we start by training the model on the entire dataset.

```
In [ ]: from sklearn.tree import DecisionTreeRegressor

regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X, y)
```

We also evaluate the model on the dataset to see how it performs.

```
In [ ]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(X)
score = mean_absolute_error(y, target_predicted)
print(f"On average, our regressor makes an error of {score:.3f} ")
```

The resulting mean absolute error is actually very impressive: it says that our model actually got the median price of each house accurately. So, the question is should we pick this model? The simple answer is not so fast! Remember we trained and evaluated our model on the same dataset and the decision tree regressor could have memorized the dataset and would therefore give us the wrong notion about its ability to generalize on unseen data.

When training our model, we are interested in it minimizing the error on unseen data, and this is the idea behind us splitting the data into train and test sets.

So, we can begin by splitting our dataset as shown below. Again, we train our model on the training set and evaluate its performance on the same data and record the error.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

We again train the model on our training data.

```
In [ ]: regressor.fit(X_train, y_train)
```

We predict the data on our training data and look at the resulting training error.

```
In [ ]: y_predicted = regressor.predict(X_train)
score = mean_absolute_error(y_train, y_predicted)
print(f"The training error of our model is {score:.3f}")
```

The output tells us that the model was able to accurately predict the actual house value. To check the validity of the model's strength, we use the test data, which has not been 'seen' by the model, to predict the house median price.

```
In [ ]: y_predicted = regressor.predict(X_test)
score = mean_absolute_error(y_test, y_predicted)
print(f"The testing error of our model is {score:.3f} ")
```

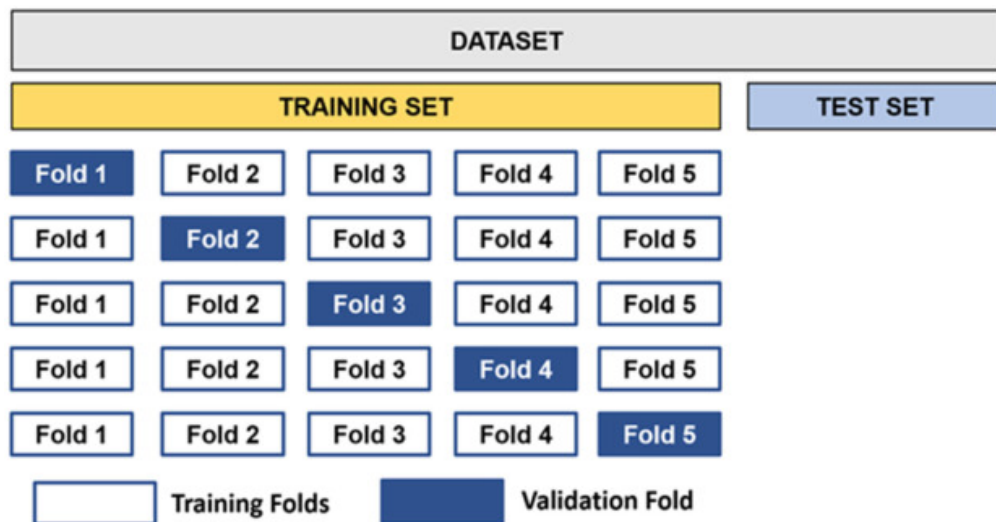
We now see that there is a bigger error on the testing dataset as compared to the training set and hence signs that our model is *overfitting*. To avoid issues with overfitting, we can use cross-validation that splits the data into smaller chunks iteratively so that it covers the entire train data, which is alternatively split into train and validation sets.

## 1.2 Cross-Validation

In this technique, the train data is split into train and validation sets multiple times, and each split is called a **Fold**. In this subsection, we will discuss the k-fold cross-validation.

K-fold cross-validation splits the data into  $k$  segments where  $k-1$  segments are used for training and the remaining segment used for validation as shown in the diagram below.

**Fig 1: K-Fold Cross Validation**



Source: Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.

Below are the steps involved in performing a cross-validation:

1. Shuffle the dataset.
2. Hold out the test data.
3. Perform k-fold cross validation on the train dataset.
4. Get the best evaluation score, which is the mean of all the folds.
5. Perform model evaluation on the test data.

Cross-validation allows us to estimate the robustness of our model by splitting the dataset repetitively a given number of times.

The next cells demonstrate how to apply cross-validation on the dataset.

```
In [ ]: from sklearn.model_selection import ShuffleSplit, cross_validate

cv = ShuffleSplit(n_splits=40, test_size=0.3, random_state=0)
cv_results = cross_validate(regressor, X, y, cv=cv, scoring="neg_mean_absolute_e
```

We convert the `cv_results` Python dictionary into a dataframe for easier visualization.

```
In [ ]: import pandas as pd

cv_results = pd.DataFrame(cv_results)
cv_results.head()
```

We negate the `test_score` to get the actual test errors because the parameter `scoring="neg_mean_absolute_error"` gives negative values of the mean absolute error.

```
In [ ]: cv_results["test_error"] = -cv_results["test_score"]
```

```
In [ ]: cv_results.head(10)
```

The plot below shows how the testing errors are distributed.

```
In [ ]: import matplotlib.pyplot as plt

plt.hist(cv_results["test_error"], density=True)
# cv_results["test_error"].plot.hist(bins=10, edgecolor="black")
plt.xlabel("Mean absolute error")
plt.ylabel("Frequency Percentage %")
plt.suptitle(
    "Fig. 2: Test Errors Distribution.", fontweight="bold", horizontalalignment=
)
plt.show()
```

```
In [ ]: print(
    f"The mean cross-validated testing error is: "
    f"{cv_results['test_error'].mean():.3f}"
)
```

The error has reduced when compared to the initial error without using cross-validation.

```
In [ ]: print(
    f"The standard deviation of the testing error is: "
    f"{cv_results['test_error'].std():.3f}"
)
```

Cross-validation reduces the error on our model albeit at a smaller fraction as compared to the case when we did not use cross-validation.

A problem tends to occur in our bid to improve model performance: we can find ourselves solving one problem (underfitting) while introducing another problem (overfitting). In the next subsection, let's discuss the overfitting and underfitting concepts.

## 1.3 Overfitting and Underfitting

When the model memorizes the train data, then we say overfitting has occurred. In such a case, we get a high evaluation score on our train dataset but a not-so-impressive score on the test data set; therefore, the model fails to generalize well on unseen data.

Models that overfit respond to random noise in the training set and thus become sensitive to small fluctuations in the data. Overfitting is a combination of low bias and high variance.

Underfitting on the other hand is characterized by a low performance score on both the training dataset and the validation dataset. Underfitting is caused by simple algorithms that are unable to learn complex patterns existing in our data. Underfitting is also referred to as high bias and often low variance.

A right fit model neither underfits nor overfits, and the model performance in the training set is comparable to the test set. Linear machine learning algorithms such as logistic regression tend to underfit on the dataset while non-linear models like polynomial regression tend to overfit a dataset.

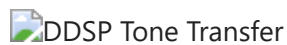
In case our model underfits, to improve its performance, we can

1. Use non-linear algorithms.
2. Increase the complexity of the model by tweaking its parameters.
3. Use non-parameterized algorithms.

To curb overfitting of our models, we

1. Increase the data size used for training to improve the generalizing ability of the model.
2. Use regularization techniques like the L1 and L2 norms.
3. Tune the hyperparameters of our model.
4. Reduce the number of features in our dataset.
5. Reduce the complexity of the model.
6. Apply cross-validation techniques.

### Fig 3: Overfitting and Underfitting Examples



Source: [Badillo, Solveig, et al. "An Introduction to Machine Learning." \*Clinical Pharmacology & Therapeutics\*, vol. 107, no. 4, 2020, pp. 871-885.](#)

In the next cells, we evaluate our model and determine whether it is a good fit, underfitting or overfitting on our dataset.

```
In [ ]: import pandas as pd
        from sklearn.model_selection import ShuffleSplit, cross_validate

        cv = ShuffleSplit(n_splits=30, test_size=0.2)
        cv_results = cross_validate(
            regressor,
            X,
            y,
            cv=cv,
            scoring="neg_mean_absolute_error",
            return_train_score=True,
            n_jobs=2,
        )
        cv_results = pd.DataFrame(cv_results)
```

As discussed in the previous subsection, we negate the scores to get the actual train and test errors.

```
In [ ]: scores = pd.DataFrame()
        scores[["train error", "test error"]] = -cv_results[["train_score", "test_score"]]
```

The next cell plots the distribution of the errors.

```
In [ ]: import matplotlib.pyplot as plt

        scores.plot.hist(bins=50, edgecolor="black")
        plt.xlabel("Mean absolute error")
        plt.suptitle(
            "Fig. 4: Train and Test Errors Distribution via Cross-validation.",
            fontweight="bold",
```

```
horizontalalignment="right",
)
plt.show()
```

We can observe that the training error is zero, implying that our model is not underfitting. The test error is larger, and therefore, our model seems to have memorized the noise in our dataset. Hence, its generalization abilities have been compromised.

One of the challenges with hyperparameter tuning is that we can easily shift from a case of a model underfitting into another case of the model overfitting on our dataset. To overcome such challenges, we need to consider **validation curves**.

## 1.4 Validation Curves

The validation curve varies the values of the hyperparameter(s) and gives the intuition of how the model behaves given different hyperparameter values.

```
In [ ]: from sklearn.model_selection import validation_curve

max_depth = [1, 3, 5, 10, 15, 20, 25, 30]
train_scores, test_scores = validation_curve(
    regressor,
    X,
    y,
    param_name="max_depth",
    param_range=max_depth,
    cv=cv,
    scoring="neg_mean_absolute_error",
    n_jobs=2,
)
train_errors, test_errors = -train_scores, -test_scores
```

In the example, we will restrict ourselves to one of the hyperparameters, the `max_depth`.

```
In [ ]: plt.plot(max_depth, train_errors.mean(axis=1), label="Training error")
plt.plot(max_depth, test_errors.mean(axis=1), label="Testing error")
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 5: Validation Curve for Decision Tree",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

From the above plot, we see that for lower values of maximum depth, the model seems to underfit while as the maximum depth of trees increases, the model overfits on our dataset. So, for the values of 10, we can say that our model will provide a good fit and hence better generalization on an unseen dataset. The plot below repeats the above plot but on training errors.

```
In [ ]: plt.errorbar(
    max_depth,
    train_errors.mean(axis=1),
    yerr=train_errors.std(axis=1),
    label="Training error",
)
plt.errorbar(
    max_depth,
    test_errors.mean(axis=1),
    yerr=test_errors.std(axis=1),
    label="Testing error",
)
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 6: Validation Curve for Decision Tree using Train Errors",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Our target when running machine learning models is to get an unbiased estimate of our errors. This is possible when we understand how the concept of bias-variance tradeoff relates to underfitting and overfitting.

## 1.5 Bias-Variance Tradeoff

We can divide the prediction errors from machine learning algorithms into two:

- Reducible parts
- Irreducible parts.

The irreducible parts are caused by stochastic noise as a result of missing important features or measurement errors and can't be reduced even with very good models. The reducible part, on the other hand, constitutes errors due to:

- Bias
- Variance

**Error due to Bias** - Bias is the difference between our predicted values and the actual values we are trying to predict. These errors occur when the algorithm is too simple (less complex) to capture the functional relationship between our features. Models having high bias tend to oversimplify the model by not learning a lot from the training data. This causes the model to make some systematic mistakes, and therefore, the predictions will be biased. This is also referred to as underfitting.

**Error due to Variance** - We define variance as the measure of spread in our dataset. These errors occur when the algorithm is so complex that it extracts patterns from the noise in the dataset. Models with high variance perform very well on training data but do

not generalize well on unseen data. This is what we will refer to as overfitting, as discussed above.

So, we see two issues here:

1. If our model is too simple and has few parameters, then there is a tendency that we risk having a model with high bias and low variance.
2. If our model is too complex with many parameters, then it is highly likely the model will overfit on the dataset due to high variance and low bias on the dataset.

This is what introduces the bias-variance tradeoff since it is impossible to have a model that will be both complex and simple at the same time.

Building accurate models requires understanding these errors so as to avoid our model underfitting or overfitting.

In the next few cells, we will demonstrate the concept of bias and variance tradeoff.

```
In [ ]: from mlxtend.evaluate import bias_variance_decomp
```

We use the data as above and the same algorithm to demonstrate the concept of bias-variance tradeoff.

```
In [ ]: from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X, y = housing.data.values, housing.target.values * 10
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=1
)
```

In the next cell, we will define the hyperparameter that will define an increase in model complexity.

```
In [ ]: max_levels = list(range(1, 50))
levels = []
squared_bias_plus_variance = []
for level in max_levels:
    model = DecisionTreeRegressor(max_depth=level)
    model.fit(X_train, y_train)
    mse, bias, var = bias_variance_decomp(
        model,
        X_train,
        y_train,
        X_test,
        y_test,
        loss="mse",
        num_rounds=200,
        random_seed=1,
    )
    score = model.score(X_test, y_test)
    squared_bias_plus_variance.append(bias**2 + var)
    levels.append(level)
```

In the next cell, we plot the bias-variance tradeoff chart that shows how an increase in model complexity affects bias and variance.

```
In [ ]: import plotly.graph_objects as go

scatter = go.Scatter(x=levels, y=squared_bias_plus_variance)
layout = go.Layout(
    title="Fig. 7. Bias variance tradeoff",
    xaxis=dict(title="levels"),
    yaxis=dict(title="bias^2+variance"),
)
go.Figure(data=[scatter], layout=layout)
```

It can be seen that as the model complexity increases, the error reduces.

Now that we have a good understanding of how to improve our model performance, we can go ahead and learn how to tweak the hyperparameters of our models.

## 2. Hyperparameter Tuning

Hyperparameter tuning involves searching over a given set of values and finding the optimal values of a hyperparameter in a machine learning model. We expect that after performing hyperparameter tuning, the result of the model training will give us a better score without overfitting on the dataset.

Now let's highlight the difference between a parameter and a hyperparameter.

While a parameter is learned from the data during model training, a hyperparameter is set by the user before we begin training our model.

An example of a parameter is the coefficients we obtain from training a linear regression model while the number of epochs and batch size are examples of hyperparameters. We should note that some models like linear regression only have parameters while others like the K-Nearest Neighbors (KNN) contain only hyperparameters.

Before commencing on to hyperparameter tuning, we first define the **hyperparameter space**, which is the set of all possible hyperparameter values. Hyperparameters can either be discrete or continuous values. When defining the hyperparameter space, we also define their underlying distribution.

In the next subsections of this lesson, we intend to discuss the various types of hyperparameter tuning.

### 2.1 Manual Search

The manual search method entails defining the search space and looping all the possible hyperparameters to get a combination that will give us the best outcome. We follow these steps when performing manual search:

1. Split the train data into train and test splits.

2. Define the initial hyperparameter values.
3. Perform k-fold cross validation on the training data.
4. Get the validation score.
5. Choose another set of hyperparameter values.
6. Repeat steps 3-5 until we get satisfactory performance.
7. Train the model on the full train set with the chosen hyperparameter value.
8. Evaluate this performance on the test set.

In the example below, we show how we can get and set the value(s) of model hyperparameters. We will consider the [blood transfusion data](#).

```
In [ ]: from sklearn import set_config

set_config(display="diagram")

import pandas as pd
```

We load the data and display the first few rows.

```
In [ ]: url = "https://archive.ics.uci.edu/ml/machine-learning-databases/blood-transfusi
data_df = pd.read_csv(url)
data_df.head()
```

The next step involves renaming the columns for easier handling.

```
In [ ]: cols = [c.lower().split()[0] for c in data_df.columns]
cols[-1] = "class_name"
data_df.columns = cols
```

```
In [ ]: data_df.head()
```

We then define the target class that will set the stage for the type of machine learning problem we are supposed to solve. In our case, we are presented with a binary classification problem that we will not delve into too much as we are only interested in learning the hyperparameter tuning techniques.

```
In [ ]: target_name = "class_name"
feature_columns = ["recency", "frequency", "monetary"]

target = data_df[target_name]
data = data_df[feature_columns]
```

We transform the data by rescaling it using `StandardScaler` that removes the mean of the feature and scales to a unit variance.

```
In [ ]: from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model = Pipeline(
    steps=[
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
```

```
]
)
```

As discussed in the previous section, we evaluate the performance of the model using cross-validation.

```
In [ ]: from sklearn.model_selection import cross_validate

cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Accuracy score via cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

In the demonstrations that will follow in this lesson, we will try to tune the hyperparameters `learning_rate` and `max_leaf_nodes` of the `HistGradientBoostingClassifier` model. We start first by displaying their default values below.

```
In [ ]: print("learning_rate default value", model.get_params()["classifier__learning_ra
print("max_leaf_nodes default value", model.get_params()["classifier__max_leaf_n
```

Without specifying the value of the hyperparameters, the model uses the default parameters, for example, the default value for `C` is 1. We can manually tune the hyperparameter, in this case, change the value of `C` as shown in the cell below.

```
In [ ]: model.set_params(classifier__learning_rate=1e-3)
model.set_params(classifier__max_leaf_nodes=20)
cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Model accuracy score with cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

The hyperparameters are stored in `model.get_params` and we can explore them one by one. For demonstration purposes we will consider tuning the hyperparameter `C` in the cells below.

The cell below displays the values of the hyperparameters we have set above.

```
In [ ]: print("learning_rate default value", model.get_params()["classifier__learning_ra
print("max_leaf_nodes default value", model.get_params()["classifier__max_leaf_n
```

In the next cell, we manually set a couple of values for `C` and choose the one that gives us better performance.

```
In [ ]: for learning_rate in [1e-3, 1e-2, 1e-1, 1, 10]:
        for max_leaf_nodes in [3, 5, 10, 15, 20, 30]:
            model.set_params(classifier__learning_rate=learning_rate)
            model.set_params(classifier__max_leaf_nodes=max_leaf_nodes)
            cv_results = cross_validate(model, data, target)
            scores = cv_results["test_score"]
```

```
print(
    f"Model accuracy score with cross-validation:\n learning_rate={learn
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

From the cell above, we see that the value of `C=0.01` gives better accuracy.

## 2.2 Grid Search

Given a hyperparameter space, grid search looks for all the possible combinations that improve the performance of the model. In this method, the underlying distribution of the hyperparameters need not be known.

So, the steps involved in grid search are as follows.

1. Split the data into train and test sets.
2. Define the hyperparameter space.
3. Construct a nested loop of the number of hyperparameters.
4. Perform cross validation on the train set and store the resulting score in each loop.
5. Train the model on the train set and evaluate it on the test set.

The output of the grid search will be equal to the output of the manual search since the model is evaluated at each pair of hyperparameters in our grid.

Just like in the manual tuning section above, we demonstrate how to apply grid search in hyperparameter tuning.

We start by splitting our dataset into training and testing sets.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(data, target, random_state=4
```

We use the histogram gradient classifier model, which is a tree-based classifier.

```
In [ ]: from sklearn.ensemble import HistGradientBoostingClassifier

model = Pipeline(
    [
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)
model
```

Grid search is costly as it finds a combination of all the hyperparameters in our hyperparameter space. In this example, we share only limited nodes, and we choose to select only two hyperparameters to tune.

```
In [ ]: from sklearn.model_selection import GridSearchCV

param_grid = {
    "classifier__learning_rate": (0.001, 0.01, 0.1, 1, 10),
```

```

        "classifier__max_leaf_nodes": (3, 5, 10, 15, 20, 30),
    }
    model_grid_search = GridSearchCV(model, param_grid=param_grid, n_jobs=2, cv=2)
    model_grid_search.fit(X_train, y_train)

```

We then evaluate the accuracy of our model using the test set.

```

In [ ]: accuracy = model_grid_search.score(X_test, y_test)
        print(f"The test accuracy score of the grid-searched pipeline is: " f"{accuracy:

```

The grid search takes the hyperparameter space with all the hyperparameter values in it and finds all the possible combinations of the hyperparameters. Like in our case where we have four learning\_rate and max\_leaf\_nodes values, there will be  $4 \times 4 = 16$  combination of hyperparameters, and we select the pair that gives us the best result. This makes the process expensive because for any value we add, the process becomes more and more computationally unsustainable. We can actually see that Grid Search is similar to the manual search we saw in the previous subsection.

From the selected hyperparameters, we can now predict the first five targets.

```

In [ ]: model_grid_search.predict(X_test.iloc[0:5])

```

We now visualize the original values in the cell below.

```

In [ ]: y_test.iloc[0:5].values

```

By using the `best_params_` attribute, we can see the selected hyperparameters that gives us the highest accuracy.

```

In [ ]: print(f"The best set of parameters is: " f"{model_grid_search.best_params_}")

```

We inspect all the values stored in `cv_results_` using a dataframe as shown below.

```

In [ ]: cv_results = pd.DataFrame(model_grid_search.cv_results_).sort_values(
        "mean_test_score", ascending=False
    )
        cv_results.head()

```

We shorten the headers to make the dataframe more readable.

```

In [ ]: # get the parameter names
        column_results = [f"param_{name}" for name in param_grid.keys()]
        column_results += ["mean_test_score", "std_test_score", "rank_test_score"]
        cv_results = cv_results[column_results]

```

```

In [ ]: def shorten_param(param_name):
        if "__" in param_name:
            return param_name.rsplit("__", 1)[1]
        return param_name

        cv_results = cv_results.rename(shorten_param, axis=1)
        cv_results.head()

```

Since we are only interested in our two hyperparameters, the table below displays the model accuracy for each pair.

```
In [ ]: pivoted_cv_results = cv_results.pivot_table(
        values="mean_test_score", index=["learning_rate"], columns=["max_leaf_nodes"]
    )

pivoted_cv_results
```

We can visualize the table above in a heatmap as shown in the next cell.

```
In [ ]: import seaborn as sns

ax = sns.heatmap(pivoted_cv_results, annot=True, cmap="YlGnBu", vmin=0.0, vmax=0.8)
ax.invert_yaxis()
plt.suptitle(
    "Fig. 8: Heatmap of the Model Results",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

The biggest challenge we have with the two methods we have seen above is that they are very computationally intensive especially when our hyperparameter space is large. In the next subsection, we will discuss the random search method that tries to solve this problem.

## 2.3 Random Search

This method randomly selects the hyperparameter for each iteration. It outperforms the grid search when the data is small or we have no idea of the range of hyperparameter space. It is also much more efficient when compared to grid search. In random search, the hyperparameter distribution must be specified together with the hyperparameter space, which is wider as compared to the grid search case.

The steps involved in random search are:

1. Split the data into train and test sets.
2. Define the number of trials and set a random seed.
3. Define the hyperparameter space with its distribution.
4. Define an iterator consisting of random hyperparameter combinations.
5. Loop through the iterator.
6. After getting the best possible combination of hyperparameters, we train the model on the full train set and evaluate the model performance.

In the following cells, we demonstrate the random search method for tuning the hyperparameters on our blood transfusion data.

```
In [ ]: model = Pipeline(
    [
        ("preprocessor", StandardScaler()),
```

```

        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)

```

We can view the default hyperparameter values in the cell below.

```

In [ ]: from pprint import pprint

print("Default parameters:\n")
pprint(model.get_params())

```

The next cell defines the hyperparameter search space.

```

In [ ]: from scipy.stats import loguniform

class loguniform_int:
    """Integer valued version of the log-uniform distribution"""

    def __init__(self, a, b):
        self._distribution = loguniform(a, b)

    def rvs(self, *args, **kwargs):
        """Random variable sample"""
        return self._distribution.rvs(*args, **kwargs).astype(int)

from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "classifier__learning_rate": loguniform(1e-6, 1e3),
    "classifier__max_leaf_nodes": loguniform_int(3, 100),
}

```

We can now train the model on our training data using cross-validation that executes 100 iterations of 5 folds.

```

In [ ]: model_random_search = RandomizedSearchCV(
    model,
    param_distributions=param_distributions,
    n_iter=10,
    cv=5,
    verbose=1,
    n_jobs=-1,
    return_train_score=True,
)
model_random_search.fit(X_train, y_train)

```

The random search method for hyperparameter tuning selects the hyperparameter combinations randomly as opposed to grid search where each value in our hyperparameter space was used explicitly. In random search, the choice of one hyperparameter value is independent from the other ones. As opposed to grid search, where our hyperparameter space consisted of hyperparameter values, random search allows the use of sampling distributions.

The next cell displays the best parameters from our model.

```
In [ ]: model_random_search.best_params_
```

We evaluate our model performance on the test dataset; however, we do not expect good results as the hyperparameters were arbitrarily chosen for the purposes of demonstrating the random search method.

```
In [ ]: accuracy = model_random_search.score(X_test, y_test)

print(f"The test accuracy score of the best model is " f"{accuracy:.3f}")
```

Just like the grid search process above, we inspect the cv results in the next cell.

```
In [ ]: cv_results_rs = pd.DataFrame(model_random_search.cv_results_).sort_values(
    "mean_test_score", ascending=False
)
cv_results_rs.head()
```

```
In [ ]: def shorten_param(param_name):
    if "__" in param_name:
        return param_name.rsplit("__", 1)[1]
    return param_name

cv_results_rs = cv_results_rs.rename(shorten_param, axis=1)
cv_results_rs.head()
```

The table below displays the pairs that the random search tuning picked by the random search algorithm. Since `n_iter=10`, random search will pick 10 pairs at random.

```
In [ ]: pivoted_cv_results_rs = cv_results_rs.pivot_table(
    values="mean_test_score", index=["learning_rate"], columns=["max_leaf_nodes"]
)

pivoted_cv_results_rs
```

We can observe that hyperparameter tuning using random search does not evaluate the model at all pairs but randomly selects the best possible combination among pairs. Since random search does not pick all possible pairs, it tends to be faster than manual search and grid search, and its accuracy is nearly equal to grid search.

## 2.4 Bayesian Optimization

Bayesian optimization improves search space based on what it learns from the previous iterations. In this kind of hyperparameter selection, we first build the probability model of the objective function and then choose the optimal hyperparameters.

There are several techniques of the Bayesian Optimization listed below.

- Gaussian Process (GP).
- Sequential model-based optimization
- Tree Parzen Estimator (TPE)

Bayesian optimization memorizes the model's past performance, which it uses to find the probability of a score given a parameter, that is,  $P(\text{score}|\text{hyperparameters})$ .

The model can be optimized easily and used to find the next hyperparameter(s).

The steps of a Bayesian optimization are:

1. Find the probability model of a score given a hyperparameter.
2. Find the optimal hyperparameters of the model in (1).
3. Plug the hyperparameters into the objective function.
4. Update the probability model given the new results.
5. Repeat steps to 1 to 4 until we get a satisfactory result.

We will discuss more on Bayesian optimization in lesson 4.

### 3. Conclusion

In this lesson, we have seen the various techniques for improving model performance and the impact of hyperparameters on model performance. We have also discussed how to automate the search for hyperparameters using grid search and randomized search. In the next lesson, we will apply these methods to a real-world example.

#### References

1. Badillo, Solveig, et al. "An Introduction to Machine Learning." *Clinical Pharmacology & Therapeutics*, vol. 107, no. 4, 2020, pp. 871-885.
2. Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.
3. Yang, Li, and Abdallah Shami. "On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice." *Neurocomputing*, vol. 415, 2020, pp. 295-316.

# MACHINE LEARNING IN FINANCE

## MODULE 7 | LESSON 2

---

# HOUSE PREDICTION AND HYPERPARAMETER TUNING

---

|                        |                            |
|------------------------|----------------------------|
| <b>Reading Time</b>    | 35 minutes                 |
| <b>Prior Knowledge</b> | Grid Search, Random Search |
| <b>Keywords</b>        | Grid search                |

---

*In the last lesson, we introduced hyperparameter tuning in machine learning and the merits and demerits of the various algorithms used. In this lesson, we will apply the skills learned to a house prediction exercise.*

## 1. Introduction

In this lesson we will use the housing data from the [Geo Data and Lab](#) website to predict the housing prices using Random Forest Regression and later on fine-tune its hyperparameters so as to improve the model performance.

We begin by first loading the relevant packages below.

```
In [ ]: # data manipulation and plotting
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# from Scikit-Learn
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

We then load the downloaded file to our working space and read it.

```
In [ ]: df = pd.read_csv("kc_house_data.csv")
```

```
In [ ]: df.head()
```

Since the data is already processed, we will lightly touch on the exploratory data analysis phase and concentrate mostly on the modeling phase.

Below we can see the size of the data and examine missing values.

## 2. Exploratory Data Analysis

```
In [ ]: print(f"The dataset contains {df.shape[0]} samples and " f"{df.shape[1]} feature
```

```
In [ ]: df.isnull().sum()
```

We then drop the id and date column from our data as they will not be useful for our prediction exercise. We also separate the target variable from the predictor variables.

```
In [ ]: X = df.drop(["id", "price", "date"], axis=1)
y = df["price"]
```

The next step is to pick out categorical variables from numerical variables using column selector from `sklearn` 's `compose`.

```
In [ ]: from sklearn.compose import make_column_selector as selector

categorical_columns_selector = selector(dtype_include=object)
categorical_columns = categorical_columns_selector(X)
categorical_columns
```

```
In [ ]: # now let's identify the numerical variables

num_vars = [var for var in X.columns if var not in categorical_columns]

# number of numerical variables
print(len(num_vars))
print(num_vars)
```

We note that some categorical variables are numerical and we can put them into their correct list.

```
In [ ]: # let's make a list of discrete variables
categorical_vars = [var for var in num_vars if len(X[var].unique()) < 20]

print("Number of categorical variables: ", len(categorical_vars))
print(categorical_vars)
```

```
In [ ]: # let's visualize the categorical variables

X[categorical_vars].head()
```

Therefore, the remaining numerical variables will now be classified as continuous variables.

```
In [ ]: # make list of continuous variables
cont_vars = [var for var in num_vars if var not in categorical_vars]

print("Number of continuous variables: ", len(cont_vars))
print(cont_vars)
```

```
In [ ]: # Let's visualize the continuous variables
```

```
X[cont_vars].head()
```

We now visualize the variables' distribution using histograms as shown below.

```
In [ ]: # Lets plot histograms for all continuous variables
```

```
X[cont_vars].hist(bins=30, figsize=(15, 15))
plt.suptitle("Fig. 1: Histogram Plot", fontweight="bold", horizontalalignment="r")
plt.show()
```

We can now standardize the continuous variables and encode the discrete variables to eliminate possible bias towards larger values.

### 3. Modeling Pipeline

```
In [ ]: from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
categorical_preprocessor = OneHotEncoder(handle_unknown="ignore")
numerical_preprocessor = StandardScaler()
```

```
In [ ]: from sklearn.compose import ColumnTransformer
```

```
preprocessor = ColumnTransformer(
    [
        ("one-hot-encoder", categorical_preprocessor, categorical_vars),
        ("standard_scaler", numerical_preprocessor, cont_vars),
    ]
)
```

We now create a pipeline of our regression model as shown below.

```
In [ ]: model = Pipeline(
    [
        ("preprocessor", preprocessor),
        ("regressor", RandomForestRegressor(random_state=42)),
    ]
)
```

```
In [ ]: from sklearn import set_config
```

```
set_config(display="diagram")
model
```

The data is now split into training data and testing data.

```
In [ ]: # Let's separate into train and test set
# Remember to set the seed (random_state for this `sklearn` function)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, # predictive variables
    y, # target
    test_size=0.3, # portion of dataset to allocate to test set
```

```

    random_state=0, # we are setting the seed here
)

X_train.shape, X_test.shape

```

We can note below that the target variable is skewed to the left and our model will have a tendency to predict values to the most common value. Therefore, we transform it to make it more normal.

```
In [ ]: # histogram to evaluate target distribution
```

```

df["price"].hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Sale price")
plt.suptitle(
    "Fig. 2: Target Variable Distribution",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

After log transforming the target variable, it now looks more normally distributed and will therefore use the log transform variable as our target.

```
In [ ]: # Let's transform the target using the Logarithm
```

```

np.log(df["price"]).hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Log of Sale Price")
plt.suptitle(
    "Fig. 3: Log Transformed Target", fontweight="bold", horizontalalignment="ri
)
plt.show()

```

```
In [ ]: y_train = np.log(y_train)
        y_test = np.log(y_test)
```

```
In [ ]: X_train.head()
```

We now train the model using the pipeline we formulated above.

```
In [ ]: _ = model.fit(X_train, y_train)
```

```
In [ ]: X_test.head()
```

We then use the model to make predictions about unseen test data and evaluate its performance.

```

In [ ]: target_predicted = model.predict(X_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(y_test, target_predicted):.3f}"
)

```

## 4. Regression Metrics

There are various metrics we can use to evaluate the performance of a regression model.

```
In [ ]: model.score(X_test, y_test)
```

The model performed really well, and we will now want to see if we can improve this performance using hyperparameter tuning.

```
In [ ]: from sklearn.dummy import DummyRegressor

dummy_regressor = DummyRegressor(strategy="mean")
dummy_regressor.fit(X_train, y_train)
print(
    f"R2 score for a regressor predicting the mean:"
    f"{dummy_regressor.score(X_test, y_test):.3f}"
)
```

```
In [ ]: from sklearn.metrics import mean_absolute_error

print(f"Mean absolute error: " f"{mean_absolute_error(y_test, target_predicted):
```

## 5. Hyperparameter Tuning

### 5.1 Random Search

We will need to define the hyperparameter grid to be able to use random search.

```
In [ ]: from pprint import pprint

# Look at parameters used by our current forest
print("Parameters currently in use:\n")
pprint(model.get_params())
```

Random search has the advantage of being fast as it does not use all the values in the hyperparameter space but rather randomly selects variables.

```
In [ ]: from sklearn.model_selection import RandomizedSearchCV

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start=10, stop=90, num=10)]
# Number of features to consider at every split
max_features = ["auto", "sqrt"]
# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(10, 50, num=21)]
max_depth.append(None)
# Minimum number of samples required to split a node
min_samples_split = [2, 5, 10]
# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 4]
# Method of selecting samples for training each tree
bootstrap = [True, False]
# Create the random grid
```

```

random_grid = {
    "regressor__n_estimators": n_estimators,
    "regressor__max_features": max_features,
    "regressor__max_depth": max_depth,
    "regressor__min_samples_split": min_samples_split,
    "regressor__min_samples_leaf": min_samples_leaf,
    "regressor__bootstrap": bootstrap,
}
pprint(random_grid)

```

We now create a model with all the parameters specified above.

```

In [ ]: model_random_search = RandomizedSearchCV(
        model,
        param_distributions=random_grid,
        n_iter=10,
        cv=5,
        verbose=1,
    )
model_random_search.fit(X_train, y_train)

```

We can now visualize the best hyperparameters that we will fit back to our model.

```

In [ ]: model_random_search.best_params_

```

By using this hyperparameter values, we now evaluate our algorithm's performance.

```

In [ ]: def evaluate(model, test_features, test_labels):
        predictions = model.predict(test_features)
        errors = abs(predictions - test_labels)
        mape = 100 * np.mean(errors / test_labels)
        accuracy = 100 - mape
        print("Model Performance")
        print("Average Error: {:.4f} dollars.".format(np.mean(errors)))
        print("Accuracy = {:.2f}%.".format(accuracy))

        return accuracy

base_model = RandomForestRegressor(n_estimators=10, random_state=42)
base_model.fit(X_train, y_train)
base_accuracy = evaluate(base_model, X_test, y_test)

```

```

In [ ]: best_random = model_random_search.best_estimator_
random_accuracy = evaluate(best_random, X_test, y_test)

```

Comparing the performance of our model without and with hyperparameter tuning show us a marginal increase in accuracy.

```

In [ ]: print(
        "Improvement of {:.2f}%.".format(
            100 * (random_accuracy - base_accuracy) / base_accuracy
        )
    )

```

## 5.2 Grid Search

We now provide a framework for performing grid search on our model hyperparameters. Note that we restricted the hyperparameters to a smaller space since the grid search algorithm is very heavy. We therefore do not expect a better performance. Students are allowed to experiment with bigger spaces and see how the model performs.

```
In [ ]: from sklearn.model_selection import GridSearchCV

# Create the parameter grid based on the results of random search
param_grid = {
    "regressor__bootstrap": [True],
    "regressor__max_depth": [int(x) for x in np.linspace(10, 20, num=6)],
    "regressor__max_features": [2, 3],
    "regressor__min_samples_leaf": [3, 4, 5],
    "regressor__min_samples_split": [8, 10, 12],
    "regressor__n_estimators": [
        int(x) for x in np.linspace(start=70, stop=100, num=11)
    ],
}
# Create a based model
rf = RandomForestRegressor()
# Instantiate the grid search model
grid_search = GridSearchCV(
    estimator=model, param_grid=param_grid, cv=3, n_jobs=-1, verbose=2
)
```

```
In [ ]: # Fit the grid search to the data
grid_search.fit(X_train, y_train)
grid_search.best_params_
```

```
In [ ]: best_grid = grid_search.best_estimator_
grid_accuracy = evaluate(best_grid, X_test, y_test)
```

We also compare the performance of the grid search algorithm with that of the original random forest algorithm.

```
In [ ]: print(
    "Improvement of {:.2f}%".format(
        100 * (grid_accuracy - base_accuracy) / base_accuracy
    )
)
```

## Conclusion

In this lesson, we have seen how to apply hyperparameter tuning to a regression problem. In the next lesson, we will study objective functions and other techniques used to evaluate machine learning models.

### References

1. Breiman, Leo. "Random Forests." *Machine Learning*, vol. 45, no. 1, 2001, pp 5-32.

2. GeoDa Data and Lab. "2014-15 Home Sales in King County, WA."  
<https://geodacenter.github.io/data-and-lab/KingCounty-HouseSales2015/>
3. Ramadhan, Muhammad Murtadha, et al. "Parameter Tuning in Random Forest based on Grid Search Method for Gender Classification based on Voice Frequency."  
*DEStech Transactions on Computer Science and Engineering*, vol. 10, 2017.

---

Copyright 2025 WorldQuant University. This content is licensed solely for personal use.  
Redistribution or publication of this material is strictly prohibited.

# MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 3

---

## MODEL EVALUATION AND REGULARIZATION

---

|                        |                                                       |
|------------------------|-------------------------------------------------------|
| <b>Reading Time</b>    | 40 minutes                                            |
| <b>Prior Knowledge</b> | Introduction to hyperparameter tuning, loss functions |
| <b>Keywords</b>        | simulated annealing                                   |

---

*In the previous lesson, we applied grid search and random search hyperparameter tuning techniques to a house prediction problem and evaluated the model performance. In this lesson, we will present insights on when to use different cross-validation strategies. We will also discuss model evaluation metrics and introduce the notion of having a baseline model to compare with our model performance.*

### 1. Model Evaluation

In the previous modules, we have encountered different learning algorithms, and in all of them, we are interested in minimizing errors when presented with never-before-seen data. To achieve this, we will discuss the following concepts in this section:

1. Choosing an appropriate cross-validation strategy.
2. Building a baseline model that acts as a reference for the actual model.
3. Nested cross-validation concepts and their evaluation.
4. Metrics used in classification and regression models.

#### 1.1 Building Baseline Models

The baseline model can be thought of as a model reference to the actual model. We should consider the following points when building a baseline model:

1. It should be a simple model that is used for comparison.
2. It should be easy to explain.
3. It should be based on the actual data.

Baseline models are important for the following reasons;

#### Understanding the Data

When we train a baseline model on our dataset, we can infer the following:

- A baseline model where prediction of our target variable is completely off indicates how difficult it will be for weaker models to get patterns in the dataset.
- It can help us point out the segment of the data that gives the model a higher error.

## Faster Modeling

Once we have a baseline model in place, building the actual model pipeline becomes easier as we understand the data better.

## Performance Benchmark

The performance of the baseline model help us to decide whether we will need a complex model or a simple one.

We can divide the baseline model into two types:

1. Simple baseline model.
2. Machine learning baseline model.

In this subsection, we will consider a simple baseline model that uses simple logic to build the baseline model.

```
In [ ]: from sklearn.datasets import fetch_california_housing

data, target = fetch_california_housing(return_X_y=True, as_frame=True)
```

We split the data using `ShuffleSplit` with 20% of the data used for validation.

```
In [ ]: from sklearn.model_selection import ShuffleSplit

cv = ShuffleSplit(n_splits=30, test_size=0.2, random_state=0)
```

We define a decision tree regression function as our model for this section.

```
In [ ]: import pandas as pd
from sklearn.model_selection import cross_validate
from sklearn.tree import DecisionTreeRegressor

regressor = DecisionTreeRegressor()
cv_results_tree_regressor = cross_validate(
    regressor, data, target, cv=cv, scoring="neg_mean_absolute_error", n_jobs=2
)

errors_tree_regressor = pd.Series(
    -cv_results_tree_regressor["test_score"], name="Decision tree regressor"
)
```

We use a dummy regressor as our baseline that predicts the mean of the target computed.

```
In [ ]: from sklearn.dummy import DummyRegressor
```

```

dummy = DummyRegressor(strategy="mean")
result_dummy = cross_validate(
    dummy, data, target, cv=cv, scoring="neg_mean_absolute_error", n_jobs=2
)
errors_dummy_regressor = pd.Series(-result_dummy["test_score"], name="Dummy regr")
errors_dummy_regressor.describe()

```

In the next cell, we prepare a table that compares the performance of the decision tree regressor and the dummy regressor.

```

In [ ]: all_errors = pd.concat(
    [errors_tree_regressor, errors_dummy_regressor],
    axis=1,
)
all_errors

```

We see from the table above that in all the splits, the decision tree regressor has fewer errors compared to the dummy regressor. In the next cell, we can visualize the distribution of errors for both regression functions.

```

In [ ]: import matplotlib.pyplot as plt
import numpy as np

bins = np.linspace(start=0, stop=1, num=80)
all_errors.plot.hist(bins=bins, edgecolor="black")
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.xlabel("Mean absolute error")
plt.suptitle(
    "Fig. 1: Cross-validation testing errors.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

## 1.2 Choosing a Cross-Validation Strategy

So far we have only used `ShuffleSplit` or `KFold` to perform cross-validation, but they may not work all the time. As such, it's important to choose a cross-validation strategy wisely, depending on the task at hand. Let's consider the example below to demonstrate why it is important to choose a cross-validation strategy carefully.

```

In [ ]: from sklearn.datasets import load_iris

data, target = load_iris(as_frame=True, return_X_y=True)

```

We now create a simple classification algorithm for this task.

```

In [ ]: from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), LogisticRegression())

```

We now use this model to train and generalize on unseen data.

```
In [ ]: from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=3)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():
```

The output above shows that the model was unable to get any prediction correct when exposed to validation data. To understand this peculiar output, we investigate the target feature.

```
In [ ]: import matplotlib.pyplot as plt

target.plot()
plt.xlabel("Sample index")
plt.ylabel("Class")
plt.yticks(target.unique())
plt.suptitle(
    "Fig. 2: Class value in target y.", fontweight="bold", horizontalalignment="
)
plt.show()
```

We can observe from the above diagram that the target column is ordered, and this causes an unexpected outcome when we use cross-validation. We will now compute the class counts for the training and validation sets in the cells below.

```
In [ ]: import pandas as pd

n_splits = 3
cv = KFold(n_splits=n_splits)

train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
```

We display this in the table below.

```
In [ ]: train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
train_cv_counts
```

We can see that when training the model, in each fold, only two of the classes are available, and thus, it becomes difficult to predict on unseen data.

```
In [ ]: test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts.index.name = "Class label"
test_cv_counts
```

In the test data, only one class, which was not available in training, is used. This makes it difficult to predict, as the class was not available when training. The cells below help visualize the above tables.

```
In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 3: Training Set Data Distribution in Folds.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

```
In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 4: Data distribution in Testing Set",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

To solve the problems above, it is a good practice to shuffle our dataset before we start the modeling phase as shown below.

```
In [ ]: cv = KFold(n_splits=3, shuffle=True, random_state=0)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():.3f}")
```

We can now see that the model accuracy has really improved to more than 95%. We can now visualize the count of class in each fold for both the training and testing samples.

```
In [ ]: train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
test_cv_counts.index.name = "Class label"
```

```
In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
```

```

    "Fig. 5: Training set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

From the bar plots above, we can observe that the classes are fairly distributed in each fold. This is also true on the testing dataset below.

```

In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 6: Testing set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

Another technique will involve splitting our dataset so as to preserve the original class counts in each validation split. This is done using stratification as shown in the cells below. The steps will remain as what we have seen in the two strategies before.

```

In [ ]: from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=3)

```

```

In [ ]: results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ± {test_score.std():.3f}")

```

```

In [ ]: train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx], target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in range(n_splits)]
)
train_cv_counts.index.name = "Class label"
test_cv_counts.index.name = "Class label"

```

```

In [ ]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 7: Training set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

```
In [ ]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 8: Testing set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

We can see that the counts in the training and testing sets are not too far from each other.

## 1.3 Nested Cross-Validation

The k-fold cross validation technique that we have discussed so far is important for hyperparameter optimization and model evaluation on the validation set. The fact that we use cross-validation on the same dataset to tune our hyperparameters and choose a model from the same process may cause the model evaluation to be biased.

A work-around to this limitation is to create an outer loop of cross-validation. This process is called **nested cross-validation** and we will demonstrate this in the cells below.

We then configure the cross-validation process below. We will use the make classification dataset.

```
In [ ]: from numpy import mean, std
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, KFold, cross_val_score
```

We then split the dataset into a train and test dataset where the train dataset will also be used to train and validate the model.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    data, target, shuffle=True, random_state=0, test_size=0.3
)
```

We then define the model to be used for this problem.

```
In [ ]: model = RandomForestClassifier(random_state=1)
```

We now configure the inner cross-validation that will be used to train the hyperparameters.

```
In [ ]: cv_inner = KFold(n_splits=3, shuffle=True, random_state=1)
```

As we have seen from Lesson 1, we define the search space for our hyperparameters to be tuned and apply grid search to get the best combination of hyperparameters.

```
In [ ]: space = dict()
space["n_estimators"] = [10, 100, 500]
space["max_features"] = [2, 4, 6]
# Search for the best hyperparameter combination
search = GridSearchCV(
    model, space, scoring="accuracy", n_jobs=1, cv=cv_inner, refit=True
)
```

We now configure the cross-validation that will help us evaluate the model.

```
In [ ]: cv_outer = KFold(n_splits=10, shuffle=True, random_state=1)
```

The next cell performs the nested cross validation on our data and we output the result.

```
In [ ]: # execute the nested cross-validation
scores = cross_val_score(
    search, X_train, y_train, scoring="accuracy", cv=cv_outer, n_jobs=-1
)
# report performance
print("Accuracy: %.3f (%.3f)" % (mean(scores), std(scores)))
```

We get the average model accuracy from the 10 iterations, and the result shows that our model has a better accuracy. But in classification, we need to go beyond accuracy as a measurement of model performance. In the next section, we learn about classification metrics.

## 2. Metrics in Machine Learning

When building machine learning models, we need to measure their performance, and understanding the type of metric to use is vital if we are to judge whether we are making progress or not. As we are all aware now, machine learning tasks can be divided into regression or classification, and they are treated differently when modeling and also when measuring their performance. We will start by looking at regression metrics.

### 2.1 Regression Metrics

Linear regression involves finding the linear relationship that exists between the independent variables and the target variable, that is,  $y_i(x_i, w) = w_0 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_m x_{im} + \epsilon_i$  where  $y_i \in \mathbb{R}$  is the target variable,  $x_i \in \mathbb{R}^m$  is the predictor variables and  $\epsilon_i$  is the random error. The random error occurs as a result of one of the following

- Choosing a wrong linear model.
- Selecting less relevant variables and omitting relevant ones.
- Measurement errors.

A good linear regression estimates the coefficient of the features,  $w_i$ , and minimizes the residual errors,  $e_i = y_i - \hat{y}_i$ . This is possible by minimizing the objective function. The objective function can take one of the forms below.

### 2.1.1 Mean Squared Error (MSE)

The mean squared error is given by  $L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$

### 2.1.2 Mean Absolute Error (MAE)

The mean absolute error is given by  $L = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$

### 2.1.3 Mean Squared Logarithmic Error

This is similar to MSE, but the dependent variable is in logarithmic scale:  $L = \frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2$

In the next cells, we will look at different objective functions in regression for measuring errors.

```
In [ ]: from sklearn.model_selection import train_test_split

data_train, data_test, target_train, target_test = train_test_split(
    data, target, shuffle=True, random_state=0
)
```

We first train the data using a decision-tree regressor and then evaluate on the test dataset. In the next cells, we look at the various metrics used to evaluate machine learning models. We start with most commonly used metric, the mean squared error.

```
In [ ]: from sklearn.metrics import mean_squared_error

regressor.fit(data_train, target_train)
target_predicted = regressor.predict(data_train)

print(
    f"Mean squared error on the training set: "
    f"{mean_squared_error(target_train, target_predicted):.3f}"
)
```

We measure the mean squared error on the test data below.

```
In [ ]: target_predicted = regressor.predict(data_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(target_test, target_predicted):.3f}"
)
```

To calculate the coefficient of determination,  $R^2$ , we calculate the `score()` function as shown below.

```
In [ ]: regressor.score(data_test, target_test)
```

We can also calculate the mean absolute error for our model as shown below.

```
In [ ]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(data_test)
```

```
print(
    f"Mean absolute error: " f"{mean_absolute_error(target_test, target_predicted)
)
```

The median absolute error is equally calculated.

```
In [ ]: from sklearn.metrics import median_absolute_error

print(
    f"Median absolute error: "
    f"{median_absolute_error(target_test, target_predicted):.3f} "
)
```

## 2.1.4 Regularization

Regularization techniques are used in instances where a regression model overfits. The objective of regularization is to add a penalty term to the objective function that regularizes the size of the parameters, that is,

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\vec{w})$$
 where  $R(\vec{w})$  is the penalty term, a hyperparameter.

The most commonly used regularization techniques are ridge regression and lasso regression.

### 1. Ridge Regression

Ridge regression adds an  $L_2$ -norm term to the objective function. The term causes weight decay to the variables. The loss function of a ridge regression is given by 
$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} \|\vec{w}\|^2$$

The  $\lambda$  hyperparameter controls the amount of decay where larger values of  $\lambda$  increase decay and thus make the model more robust to multi-collinearity. Note that for the value of  $\lambda = 0$ , ridge regression is equivalent to linear regression. An increase in the value of  $\lambda$  makes the coefficients of our model approach zero.

We start by importing the libraries for this demonstration.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import mean_squared_error

sns.set_style("darkgrid")
```

We import and split the dataset into train and test sets.

```
In [ ]: from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(as_frame=True, return_X_y=True)
X.head()
```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=25
)

```

We build the ridge model below and fit it to the training set.

```

In [ ]: from sklearn.linear_model import Ridge
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import PolynomialFeatures, StandardScaler

        steps = [
            ("scalar", StandardScaler()),
            ("poly", PolynomialFeatures(degree=2)),
            ("model", Ridge()),
        ]

        ridge_pipe = Pipeline(steps)
        ridge_pipe.fit(X_train, y_train)
        # Predicting the Test set results
        y_pred = ridge_pipe.predict(X_test)

```

We apply k-Fold cross-validation on the dataset.

```

In [ ]: from sklearn.model_selection import cross_val_score

        accuracies = cross_val_score(estimator=ridge_pipe, X=X_train, y=y_train, cv=10)
        print("accuracy mean", accuracies.mean())
        print("accuracy standard deviation", accuracies.std())

```

We apply grid search to find the best model and the best alpha parameter.

```

In [ ]: from sklearn.metrics import make_scorer
        from sklearn.model_selection import GridSearchCV

        parameters = [{"model__alpha": np.arange(0, 0.2, 0.01)}]

        scoring_func = make_scorer(mean_squared_error)
        grid_search = GridSearchCV(
            estimator=ridge_pipe, param_grid=parameters, scoring=scoring_func, cv=10, n_
        )
        grid_search = grid_search.fit(X_train, y_train)

```

In the cell below, we display the alpha values that we intend to tune.

```

In [ ]: grid_search.cv_results_["params"]

```

As discussed in the model evaluation section, we will evaluate the performance of ridge regression on the dataset.

```

In [ ]: from sklearn.linear_model import Lasso, LinearRegression
        from sklearn.model_selection import (
            GridSearchCV,
            KFold,
            RepeatedKFold,
            cross_validate,

```

```

        train_test_split,
    )
    from sklearn.preprocessing import PolynomialFeatures, StandardScaler

```

We apply linear regression in the dataset and generate prediction on the test set.

```

In [ ]: lreg = LinearRegression()
        lreg.fit(X_train, y_train)

        lreg_y_pred = lreg.predict(X_test)

        # calculating Mean Squared Error (MSE)
        mean_squared_error = np.mean((lreg_y_pred - y_test) ** 2)
        print("Mean squared Error on test set : ", mean_squared_error)

        # Putting together the coefficient and their corresponding variable names
        lreg_coefficient = pd.DataFrame()
        lreg_coefficient["Columns"] = X_train.columns
        lreg_coefficient["Coefficient Estimate"] = pd.Series(lreg.coef_)
        print(lreg_coefficient)

```

In the next cell, we display the feature coefficients.

```

In [ ]: # plotting the coefficient score
        fig, ax = plt.subplots(figsize=(20, 10))

        ax.bar(
            lreg_coefficient["Columns"],
            lreg_coefficient["Coefficient Estimate"],
        )

        ax.spines["bottom"].set_position("zero")

        plt.style.use("ggplot")
        plt.suptitle(
            "Fig. 9: Coefficients Scores.", fontweight="bold", horizontalalignment="right"
        )
        plt.show()

```

```

In [ ]: # import ridge regression from `sklearn` library
        from sklearn.linear_model import Ridge

        # Train the model
        ridgeR = Ridge(alpha=1)
        ridgeR.fit(X_train, y_train)
        y_pred = ridgeR.predict(X_test)

        # calculate mean square error
        mean_squared_error_ridge = np.mean((y_pred - y_test) ** 2)
        print(mean_squared_error_ridge)

        # get ridge coefficient and print them
        ridge_coefficient = pd.DataFrame()
        ridge_coefficient["Columns"] = X_train.columns
        ridge_coefficient["Coefficient Estimate"] = pd.Series(ridgeR.coef_)
        print(ridge_coefficient)

```

## 2. Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) is another regularization model that adds an  $L_1$ -term to the objective function. It works by equating irrelevant parameters to zero. The loss function of a lasso regression is given by

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} \|w\|_1$$

Just like in ridge regression,  $\lambda$  controls the penalty's strength. Just like ridge regression above, when  $\lambda = 0$ , we have a linear regression equation. As the value of  $\lambda$  increases shrinkage to unimportant features and some will be equal to zero, the variance of the model reduces and bias increases.

In the cells below, we demonstrate how to apply lasso on our dataset.

```
In [ ]: steps = [("scalar", StandardScaler()), ("model", Lasso())

lasso_pipe = Pipeline(steps)
```

We apply cross-validation and print out the mean absolute error and the best alpha value.

```
In [ ]: cv = RepeatedKfold(n_splits=10, n_repeats=3, random_state=1)
model__alpha = np.linspace(0.1, 2, 21)
lasso = Lasso()
grid = dict()
grid["model__alpha"] = model__alpha
gscv = GridSearchCV(
    lasso_pipe, grid, scoring="neg_mean_absolute_error", cv=cv, n_jobs=-1
)
results = gscv.fit(X, y)
print("MAE: %.5f" % results.best_score_)
print("Config: %s" % results.best_params_)
```

We can now import the libraries and compute the model metric below.

```
In [ ]: from sklearn.linear_model import Lasso

# Train the model
lasso = Lasso(alpha=1)
lasso.fit(X_train, y_train)
y_pred1 = gscv.predict(X_test)

# Calculate Mean Squared Error
mean_squared_error = np.mean((y_pred1 - y_test) ** 2)
print("Mean squared error on test set", mean_squared_error)
lasso_coeff = pd.DataFrame()
lasso_coeff["Columns"] = X_train.columns
lasso_coeff["Coefficient Estimate"] = pd.Series(lasso.coef_)

print(lasso_coeff)
```

From the results above, we see the coefficients of features not relevant rounded down to zero.

## 2.2 Classification Metrics

In the next cells, we are going to discuss and demonstrate different classification metrics on the iris dataset.

```
In [ ]: from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split

        data, target = load_iris(as_frame=True, return_X_y=True)

        X_train, X_test, y_train, y_test = train_test_split(
            data, target, shuffle=True, random_state=0, test_size=0.3
        )

        model = RandomForestClassifier(random_state=1)
        model.fit(X_train, y_train)
        target_predicted = model.predict(X_test)
```

## Accuracy

We use accuracy as a baseline. Accuracy simply finds the proportion of classes we classified correctly, that is,

```
In [ ]: y_test == target_predicted
```

To get the accuracy of our model, we get the mean of the number of times we predicted correctly as shown below.

```
In [ ]: import numpy as np

        np.round(np.mean(y_test == target_predicted), 3)
```

This is implemented in `sklearn` using the `accuracy_score` as shown below.

```
In [ ]: from sklearn.metrics import accuracy_score

        accuracy = accuracy_score(y_test, target_predicted)
        print(f"Accuracy: {accuracy:.3f}")
```

## Confusion Matrix

Accuracy might not be a good measure of a model performance, especially when the target variable is imbalanced. The most common class will always have more weight. To get the finer details of the model performance, it is recommended to construct a confusion matrix to independently determine what error is for each class. The cell below shows how to construct a confusion matrix

```
In [ ]: from sklearn.metrics import ConfusionMatrixDisplay

        _ = ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
        plt.suptitle(
            "Fig. 10: Confusion Matrix.", fontweight="bold", horizontalalignment="right"
        )
        plt.show()
```

The diagonal entries represent the correct predictions and the off-diagonal entries are the incorrect predictions. For more on confusion matrices, we ask students to read [A Review on Evaluation Metrics for Data Classification Evaluations](#).

### 3. Conclusion

In this lesson we discussed simulated annealing as one of the techniques for hyperparameter tuning and also introduced objective functions in machine learning. In the next lesson we are going to see how we can apply Bayesian hyperparameter tuning in a bitcoin trading strategy.

#### References

1. Ampatzis, Christos, and Dario Izzo. "Machine Learning Techniques for Approximation of Objective Functions in Trajectory Optimisation." Proceedings of the IJCAI-09 workshop on Artificial Intelligence in Space, 2009.
2. Botchkarev, Alexei. "Evaluating Performance of Regression Machine Learning Models Using Multiple Error Metrics in Azure Machine Learning Studio." 2018.
3. Hossin, Mohammad, and Md Nasir Sulaiman. "A Review on Evaluation Metrics for Data Classification Evaluations." *International Journal of Data Mining & Knowledge Management Process*, vol 5, no. 2, 2015.

---

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

# MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 4

---

## BITCOIN TRADING STRATEGY

---

|                        |                                                                               |
|------------------------|-------------------------------------------------------------------------------|
| <b>Reading Time</b>    | 30 minutes                                                                    |
| <b>Prior Knowledge</b> | Hyperparameter tuning, supervised machine learning, classification algorithms |
| <b>Keywords</b>        | Bayesian optimization, signal                                                 |

---

*In the last lesson, we discussed objective functions and extended our knowledge on hyperparameter tuning. In this lesson, we will train classification algorithms on a Bitcoin trading strategy problem and improve the performance of one machine learning model by tuning its parameters using grid search and Bayesian optimization techniques.*

### 1. Introduction

In this problem we want to predict when to buy or sell Bitcoin. We will start by defining a *buy or sell signal* that will be represented by 1 or 0. We arrive at the signal by comparing the price trend of short-term and long-term behavior, that is, a short-term moving average greater than a long-term moving average. Then, we buy bitcoin; otherwise, we sell bitcoin. This is, therefore, a classification problem where we are interested in getting the direction of Bitcoin movement right. We will split our dataset into train and validation sets in the ratio of 4:1.

### 2. Loading Helper Packages and Data

#### 2.1 Loading Helper Packages

We load the Python packages that will assist us in achieving our objective.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set()
```

```
In [ ]: from time import time

import joblib
```

```

# display all columns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)

from bayes_opt import BayesianOptimization

# Libraries for Deep Learning Models
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
)
from sklearn.linear_model import LogisticRegression

# needed for `HistGradientBoostingClassifier`
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    StratifiedKFold,
    cross_val_score,
    cross_validate,
    train_test_split,
)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# from `keras.optimizers` import SGD

```

## 2.2 Data Loading

We fetch Bitcoin data price from Yahoo finance for the last five years.

```

In [ ]: # To fetch financial data
import yfinance as yf

# Set the ticker as 'BTC-USD'
BTC_Ticker = yf.Ticker("BTC-USD")
BTC_Data = BTC_Ticker.history(period="5y")

```

```

In [ ]: # BTC_Ticker = yf.Ticker("BTC-USD")
# BTC_Data = BTC_Ticker.history(period="max")
BTC_Data.head()

```

### 3. Exploratory Data Analysis (EDA)

This stage explores the data to help us see trends in the data.

```
In [ ]: BTC_Data.shape
```

```
In [ ]: BTC_Data.info()
```

```
In [ ]: BTC_Data.describe()
```

```
In [ ]: # Checking for any null values and removing the null values'''  
print("Null Values =", BTC_Data.isnull().values.any())
```

### 4. Data Preparation

We start by creating our target variable, that is, the buy or sell signal. The target variable constitutes our trading strategy. When the shorter term moving average goes above the longer term moving average, then it is an indicator to buy and the vice versa is also true. Read more here: [Picking buy-sell signals: A practitioner's perspective on key technical indicators for selected Indian firms.](#)

```
In [ ]: # Create short simple moving average over the short window  
BTC_Data["short_moving_avg"] = (  
    BTC_Data["Close"].rolling(window=10, min_periods=1, center=False).mean()  
)  
  
# Create Long simple moving average over the Long window  
BTC_Data["long_maving_avg"] = (  
    BTC_Data["Close"].rolling(window=60, min_periods=1, center=False).mean()  
)  
  
# Create signals  
BTC_Data["signal"] = np.where(  
    BTC_Data["short_moving_avg"] > BTC_Data["long_maving_avg"], 1.0, 0.0  
)
```

```
In [ ]: BTC_Data.head()
```

## 4.1 Feature Engineering

We now create additional features in our dataset that will help improve our model performance. The additional features are:

1. Exponential Moving Average: This gives us the price trend of the data.
2. Relative Strength Indicator (RSI): RSI measures the change in prices in a recent time frame.
3. Rate of Change: This measures the percentage change between the stock's current price and past prices.
4. Stochastic Oscillator: This compares the current closing price of a stock with its previous closing prices.

```
In [ ]: # calculation of exponential moving average

def EMA(df, n):
    EMA = pd.Series(df["Close"].ewm(span=n, min_periods=n).mean(), name="EMA_" +
                    return EMA

BTC_Data["EMA10"] = EMA(BTC_Data, 10)
BTC_Data["EMA30"] = EMA(BTC_Data, 30)
BTC_Data["EMA200"] = EMA(BTC_Data, 200)
BTC_Data.head()

# calculation of rate of change
def RC(df, n):
    M = df.diff(n - 1)
    N = df.shift(n - 1)
    RC = pd.Series(((M / N) * 100), name="RC_" + str(n))
    return RC

BTC_Data["RC10"] = RC(BTC_Data["Close"], 10)
BTC_Data["RC30"] = RC(BTC_Data["Close"], 30)

# Calculation of price momentum
def MOM(df, n):
    MOM = pd.Series(df.diff(n), name="Momentum_" + str(n))
    return MOM

BTC_Data["MOM10"] = MOM(BTC_Data["Close"], 10)
BTC_Data["MOM30"] = MOM(BTC_Data["Close"], 30)

# calculation of relative strength index
def RSI(series, period):
    delta = series.diff().dropna()
    u = delta * 0
    d = u.copy()
    u[delta > 0] = delta[delta > 0]
    d[delta < 0] = -delta[delta < 0]
```

```

u[u.index[period - 1]] = np.mean(u[:period]) # first value is sum of avg ga
u = u.drop(u.index[: (period - 1)])
d[d.index[period - 1]] = np.mean(d[:period]) # first value is sum of avg Lo
d = d.drop(d.index[: (period - 1)])
rs = (
    u.ewm(com=period - 1, adjust=False).mean()
    / d.ewm(com=period - 1, adjust=False).mean()
)
return 100 - 100 / (1 + rs)

BTC_Data["RSI10"] = RSI(BTC_Data["Close"], 10)
BTC_Data["RSI30"] = RSI(BTC_Data["Close"], 30)
BTC_Data["RSI200"] = RSI(BTC_Data["Close"], 200)

# calculation of stochastic oscillator.

def STOK(close, low, high, n):
    STOK = (
        (close - low.rolling(n).min()) / (high.rolling(n).max() - low.rolling(n)
    ) * 100
    return STOK

def STOD(close, low, high, n):
    STOK = (
        (close - low.rolling(n).min()) / (high.rolling(n).max() - low.rolling(n)
    ) * 100
    STOD = STOK.rolling(3).mean()
    return STOD

BTC_Data["%K10"] = STOK(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 10)
BTC_Data["%D10"] = STOD(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 10)
BTC_Data["%K30"] = STOK(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 30)
BTC_Data["%D30"] = STOD(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 30)
BTC_Data["%K200"] = STOK(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 200)
BTC_Data["%D200"] = STOD(BTC_Data["Close"], BTC_Data["Low"], BTC_Data["High"], 200)

```

We can display the dataframe to look at the features we have just feature-engineered.

```
In [ ]: BTC_Data.tail()
```

In the next cell, we calculate the moving average.

```
In [ ]: def MA(df, n):
    MA = pd.Series(df["Close"].rolling(n, min_periods=n).mean(), name="MA_" + str(n))
    return MA

BTC_Data["MA21"] = MA(BTC_Data, 21)
BTC_Data["MA63"] = MA(BTC_Data, 63)
BTC_Data["MA252"] = MA(BTC_Data, 252)
BTC_Data.tail()
```

We can now exclude features that are irrelevant for our prediction task such as the Bitcoin `High`, `Low` and `Open` values. Since we have already defined our trading

signals, we drop the short and long moving average features.

```
In [ ]: # excluding columns that are not needed for our prediction.

BTC_Data = BTC_Data.drop(
    [
        "High",
        "Low",
        "Open",
        "short_moving_avg",
        "long_maving_avg",
        "Dividends",
        "Stock Splits",
    ],
    axis=1,
)
```

```
In [ ]: BTC_Data = BTC_Data.dropna(axis=0)
```

We can now visualize the Bitcoin price trend over the four years as shown in the figure below.

```
In [ ]: BTC_Data[["Close"]].plot(grid=True)
plt.suptitle(
    "Fig. 1: Bitcoin Price trend", fontweight="bold", horizontalalignment="right"
)
plt.show()
```

In the cell below, we display the distribution of features that helps the modeler determine how to treat the individual features in our prediction dataset.

```
In [ ]: # histograms
BTC_Data.hist(sharex=False, sharey=False, xlabelsize=1, ylabelsize=1, figsize=(1
plt.suptitle(
    "Fig. 2: Variable Distribution", fontweight="bold", horizontalalignment="rig
)
plt.show()
```

Our interest as stated in the introduction is to predict when to buy or sell a Bitcoin. We draw a pie chart to visualize the proportion of sell to buy signals in our dataset.

```
In [ ]: fig = plt.figure()
plot = (
    BTC_Data.groupby("signal")
    .size()
    .plot(kind="pie", figsize=(6, 6), autopct="%1.1f%", y="Signal")
)
plt.suptitle(
    "Fig. 3: A Pie Chart of buy and sell signal",
    fontweight="bold",
    horizontalalignment="right",
)
plt.legend(["sell", "buy"])
plt.show()
```

The count of this signals can also be seen below.

```
In [ ]: BTC_Data.groupby(["signal"]).size()
```

We can now look at the relationship between our variables using a correlation matrix of the returns.

```
In [ ]: # correlation
BTC_data_returns = BTC_Data.shift(1)
correlation = BTC_data_returns.corr()
plt.figure(figsize=(15, 15))
plt.title("Correlation Matrix")
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="RdBu")
plt.suptitle(
    "Fig. 4: Correlation Matrix of BITCOIN return",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

## 5. Modeling

Now that we have our dataset in place, we split our dataset into a training and a holdout set, in our case the test set that will be used to backtest on our model.

### 5.1 Train test Split

We start our modeling by splitting the dataset into train and test sets in the ratio of 80:20.

```
In [ ]: def get_holdout_set(target, features, period=6):
    idx = pd.IndexSlice
    label = target.name
    dates = np.sort(y.index.get_level_values("Date").unique())
    cv_start, cv_end = dates[0], dates[-period - 2]
    holdout_start, holdout_end = dates[-period - 1], dates[-1]

    df = features.join(target.to_frame())
    train = df.loc[idx[:, cv_start:cv_end], :]
    y_train, X_train = train[label], train.drop(label, axis=1)

    test = df.loc[idx[:, holdout_start:holdout_end], :]
    y_test, X_test = test[label], test.drop(label, axis=1)
    return y_train, X_train, y_test, X_test
```

```
In [ ]: subset_validation = BTC_Data[1301:]
```

```
In [ ]: subset_validation.shape
```

```
In [ ]: # split out validation dataset for the end
subset_dataset = BTC_Data[:1300]
y = subset_dataset["signal"]
X = subset_dataset.loc[:, BTC_Data.columns != "signal"]
X.index = pd.to_datetime(X.index)
validation_size = 0.2
```

```
seed = 1
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=validation_size, random_state=1
)
```

```
In [ ]: subset_dataset.shape
```

We create a **validation** dataset that consists of the remaining data we did not pick in the subset dataset. This validation dataset will only be used in backtesting our models.

```
In [ ]: subset_validation = BTC_Data[1300:]
y_validation = subset_validation["signal"]
X_validation = subset_validation.loc[:, BTC_Data.columns != "signal"]
```

```
In [ ]: X_validation.shape
```

## 5.1 Baseline Classifier

As discussed in the previous module, it is important to train a baseline classifier that will be used as a baseline to compare with the real classifiers we will use for this task. We will use `sklearn DummyClassifier` as our baseline classifier and choose `strategy=stratified` that predicts the most frequent class.

We start by defining our evaluation metrics.

```
In [ ]: # test options for classification
num_folds = 10
seed = 42
scoring = "accuracy"
metrics = {
    "balanced_accuracy": "Accuracy",
    "roc_auc": "AUC",
    "neg_log_loss": "Log Loss",
    "f1_weighted": "F1",
    "precision_weighted": "Precision",
    "recall_weighted": "Recall",
}
```

We define a cross-validation function that will help us evaluate different algorithms.

```
In [ ]: def run_cv(clf, X=X, y=y, metrics=metrics, cv=10, fit_params=None, n_jobs=-1):
    start = time()
    scores = cross_validate(
        estimator=clf,
        X=X,
        y=y,
        scoring=list(metrics.keys()),
        cv=cv,
        return_train_score=True,
        n_jobs=n_jobs,
        verbose=1,
        fit_params=fit_params,
    )
    duration = time() - start
    return scores, duration
```

We also define a function that will help us manipulate the cross-validation output and plot them as will be seen in examples applied in this lesson.

```
In [ ]: def stack_results(scores):
    columns = pd.MultiIndex.from_tuples(
        [tuple(m.split("_", 1)) for m in scores.keys()], names=["Dataset", "Metric"]
    )
    data = np.array(list(scores.values())).T
    df = pd.DataFrame(data=data, columns=columns).iloc[:, 2:]
    results = pd.melt(df, value_name="Value")
    results.Metric = results.Metric.apply(lambda x: metrics.get(x))
    results.Dataset = results.Dataset.str.capitalize()
    return results
```

We finally define a plot function that will display the metrics of our models as box plots so as to allow us to visualize the distribution of the model performance.

```
In [ ]: def plot_result(df, model=None, fname=None):
    m = list(metrics.values())
    g = sns.catplot(
        x="Dataset",
        y="Value",
        hue="Dataset",
        col="Metric",
        data=df,
        col_order=m,
        order=["Train", "Test"],
        kind="box",
        col_wrap=3,
        sharey=False,
        height=4,
        aspect=1.2,
    )
    df = df.groupby(["Metric", "Dataset"]).Value.mean().unstack().loc[m]
    for i, ax in enumerate(g.axes.flat):
        s = f"Train: {df.loc[m[i], 'Train']:>7.4f}\nTest: {df.loc[m[i], 'Test']:>7.4f}"
        ax.text(
            0.05,
            0.85,
            s,
            fontsize=10,
```

```

        transform=ax.transAxes,
        bbox=dict(facecolor="white", edgecolor="grey", boxstyle="round,pad=0
    )
    g.fig.suptitle(model, fontsize=16)
    g.fig.subplots_adjust(top=0.9)
    if fname:
        g.savefig(fname, dpi=300);

```

We now start by defining the dummy classifier as shown below.

```

In [ ]: dummy_clf = DummyClassifier(strategy="stratified", random_state=42)
        algo = "dummy_clf"

```

We run the dummy classifier using cross-validation with the defined metrics used to measure model performance.

```

In [ ]: fname = f"{algo}.joblib"

        dummy_cv_result, run_time_algo = run_cv(dummy_clf)
        joblib.dump(dummy_cv_result, fname)

```

We can now create a table with the model performance on each metric as shown below.

```

In [ ]: dummy_result = stack_results(dummy_cv_result)
        dummy_result.groupby(["Metric", "Dataset"]).Value.mean().unstack()

```

Clearly, the model performance is kind of guess work as the model is half-right nearly all the time with its model metrics showing a very naïve model, as we expected. This performance will act as our baseline, and when building a model, we strive to get a model that outperforms the dummy classifier. In the next cell, we visualize the above metrics using box plots.

```

In [ ]: plot_result(dummy_result, model="Dummy Classifier")
        plt.suptitle(
            "Fig. 5: Dummy Classifier Metrics", fontweight="bold", horizontalalignment="
        )
        plt.show()

```

We can now starting searching for better models that will be used to beat the dummy classifier.

## 5.2 Defining the Classification Algorithms

We start by defining several models that will be evaluated and we select one of them.

```

In [ ]: # spot check the algorithms
        models = []
        models.append(("LR", LogisticRegression(n_jobs=-1)))
        models.append(("LDA", LinearDiscriminantAnalysis()))
        models.append(("KNN", KNeighborsClassifier()))
        models.append(("CART", DecisionTreeClassifier()))
        models.append(("NB", GaussianNB()))
        # Neural Network
        models.append(("NN", MLPClassifier()))

```

```
# Ensemble Models
# Boosting methods
models.append(("AB", AdaBoostClassifier()))
models.append(("GBM", GradientBoostingClassifier()))
# Bagging methods
models.append(("RF", RandomForestClassifier(n_jobs=-1)))
```

We now evaluate these models' performance using `kFold` cross-validation and record their mean and variance.

```
In [ ]: results = []
names = []
for name, model in models:
    kfold = KFold(n_splits=num_folds)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

Looking at the outputs recorded above, Random Forest Classifier outperforms the other models, and we select it as our preferred model. We can also visualize the performance of the algorithms in a box plot as shown below.

```
In [ ]: # compare algorithms
fig = plt.figure()
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
fig.set_size_inches(15, 8)
plt.suptitle(
    "Fig. 6: Algorithm Comparison", fontweight="bold", horizontalalignment="right"
)
plt.show()
```

## 6. Hyperparameter Tuning

Since random forest was our best performer, we seek to tune its hyperparameters to improve the performance even further.

### 6.1 Grid Search

```
In [ ]: # Grid Search: Random Forest Classifier

scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
n_estimators = [20, 80]
max_depth = [5, 10]
criterion = ["gini", "entropy"]
param_grid = dict(n_estimators=n_estimators, max_depth=max_depth, criterion=criterion)
model = RandomForestClassifier(n_jobs=-1)
kfold = KFold(n_splits=num_folds)
grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=cv)
grid_result = grid.fit(rescaledX, y_train)
```

```
# Print Results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_["mean_test_score"]
stds = grid_result.cv_results_["std_test_score"]
params = grid_result.cv_results_["params"]
ranks = grid_result.cv_results_["rank_test_score"]
for mean, stdev, param, rank in zip(means, stds, params, ranks):
    print("#%d %f (%f) with: %r" % (rank, mean, stdev, param))
```

We can now fix the best hyperparameters to our model and use it to compare with the dummy classifier. We repeat the same steps as in the dummy classifier problem.

```
In [ ]: # prepare model
model = RandomForestClassifier(
    criterion="gini", n_estimators=80, max_depth=10, n_jobs=-1
) # `rbf` is default kernel
```

```
In [ ]: algo_forest = "random_forest"
```

```
In [ ]: fname = f"{algo_forest}.joblib"

random_forest_cv_result, run_time_algo_forest = run_cv(model)
joblib.dump(random_forest_cv_result, fname)
```

We can now display the model performance table below.

```
In [ ]: rf_result = stack_results(random_forest_cv_result)
rf_result.groupby(["Metric", "Dataset"]).Value.mean().unstack()
```

The cell below displays the above metrics as box plots.

```
In [ ]: plot_result(rf_result, model="Random Forest Classifier")
plt.suptitle(
    "Fig. 7: Random Forest Classifier Metrics",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Clearly, random forest classifier outperforms the dummy classifier by far, but we seem to face a problem of overfitting. In the prediction task, we would like to look at ways of mitigating this. The table below shows a comparison of the random forest classifier and the dummy classifier.

```
In [ ]: results = {
    "Baseline": dummy_result,
    "Random Forest": rf_result,
}
df = pd.DataFrame()
for model, result in results.items():
    df = pd.concat(
        [
            df,
            result.groupby(["Metric", "Dataset"])
                .Value.mean()
                .unstack()["Test"]
        ]
    )
```

```

        .to_frame(model),
    ],
    axis=1,
)

df.T.sort_values("AUC", ascending=False)

```

Having settled on random forest, we can now train it on the data and analyze its performance on the holdout dataset.

```

In [ ]: model = RandomForestClassifier(
        criterion="gini", n_estimators=80, max_depth=10, n_jobs=-1
    )
model.fit(X_train, y_train)
# estimate accuracy on validation set
predictions = model.predict(X_test)
print(accuracy_score(y_test, predictions))
print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))

```

The model shows a better prediction potential on the holdout dataset with performance being upwards of 94%.

To evaluate the model's performance, we can draw the confusion matrix that allows us to visualize the models' recall, specificity, accuracy, and precision.

```

In [ ]: df_cm = pd.DataFrame(
        confusion_matrix(y_test, predictions),
        columns=np.unique(y_test),
        index=np.unique(y_test),
    )
df_cm.index.name = "Actual"
df_cm.columns.name = "Predicted"
sns.heatmap(df_cm, cmap="Blues", annot=True, annot_kws={"size": 16}) # font size
plt.suptitle("Fig. 8: Confusion Matrix", fontweight="bold", horizontalalignment="right")
plt.show()

```

We determine the variables with more predictive powers as shown below.

```

In [ ]: Importance = pd.DataFrame(
        {"Importance": model.feature_importances_ * 100}, index=X.columns
    )
Importance.sort_values("Importance", axis=0, ascending=True).plot(
    kind="barh", color="r", figsize=(12, 8)
)
plt.xlabel("Variable Importance")
plt.suptitle(
    "Fig. 9: Variable Importance", fontweight="bold", horizontalalignment="right"
)
plt.show()

```

## 6.2 Bayesian Optimization

The Bayesian optimization process uses the Gaussian process to model the mean and variance of our objective function. In every iteration, it tries different combinations of

hyperparameter values and builds a new model by using the model's past information. This has the advantage of reducing training time to achieve better results.

We start by defining the stratified `kFold` function that will be used to perform cross-validation on the training dataset.

```
In [ ]: def stratified_kfold_score(clf, X_train, y_train, n_fold):
    X, y = X_train.values, y_train.values
    strat_kfold = StratifiedKFold(n_splits=n_fold, shuffle=True, random_state=1)
    accuracy_list = []

    for train_index, test_index in strat_kfold.split(X, y):
        x_train_fold, x_test_fold = X[train_index], X[test_index]
        y_train_fold, y_test_fold = y[train_index], y[test_index]
        clf.fit(x_train_fold, y_train_fold)
        preds = clf.predict(x_test_fold)
        accuracy_test = accuracy_score(preds, y_test_fold)
        accuracy_list.append(accuracy_test)

    return np.array(accuracy_list).mean()
```

We also define a function with the model hyperparameters we would like to tune, that is, `max_samples`, `max_features` and `n_estimators`.

```
In [ ]: def bo_params_rf(max_samples, n_estimators, max_features):
    params = {
        "max_samples": max_samples,
        "max_features": max_features,
        "n_estimators": int(n_estimators),
    }
    clf = RandomForestClassifier(
        max_samples=params["max_samples"],
        max_features=params["max_features"],
        n_estimators=params["n_estimators"],
    )
    score = stratified_kfold_score(clf, X_train, y_train, 5)
    return score
```

In the cell that follows, we give a range of hyperparameter space that the Bayesian optimization will search.

```
In [ ]: rf_bo = BayesianOptimization(
    bo_params_rf,
    {"max_samples": (0.2, 1), "max_features": (0.5, 1), "n_estimators": (50, 200)}
)
```

We can now apply Bayesian optimization to search through the hyperparameter space and output the results at each iteration. The cell below outputs the performance for each hyperparameter combination it finds and labels in blue the combination with a higher score than the previous combinations it has already selected.

```
In [ ]: results = rf_bo.maximize(n_iter=200, init_points=20, acq="ei")
```

We can now display the best hyperparameters that have been encountered by Bayesian optimization.

```
In [ ]: params = rf_bo.max["params"]
params["n_estimators"] = int(params["n_estimators"])
print(params)
```

We can now fix these hyperparameters on our model as shown below.

```
In [ ]: rf_v1 = RandomForestClassifier(
    max_samples=params["max_samples"],
    max_features=params["max_features"],
    n_estimators=params["n_estimators"],
)
```

We now use our training dataset to train the above model having the best selected hyperparameters.

```
In [ ]: rf_v1.fit(X_train, y_train)
```

From the model we have trained above, we use that to predict on the holdout dataset and evaluate the model performance using the classification report.

```
In [ ]: preds = rf_v1.predict(X_test)
```

```
In [ ]: print(classification_report(preds, y_test))
```

The classification report above shows that the model correctly predicts the signals with a high precision. We can print out the values of precision score, recall score, and f1 score to better shed light on the classification report.

```
In [ ]: print("Precision:{}".format(precision_score(preds, y_test)))
print("Recall:{}".format(recall_score(preds, y_test)))
print("F1 Score:{}".format(f1_score(preds, y_test)))
```

As alluded, we get high performance on the classification report. Can we improve the model performance? Let's implement the stratified `kFold` validation below.

```
In [ ]: cv = StratifiedKFold(n_splits=5, random_state=123, shuffle=True)
scores = []

for (train, test), i in zip(cv.split(X_train, y_train), range(5)):
    rf_v1.fit(X.iloc[train], y.iloc[train])
    preds = rf_v1.predict(X_train.iloc[test])
    accuracy = accuracy_score(preds, y_train.iloc[test])
    scores.append(accuracy)

df_val = pd.DataFrame(scores, columns=["Accuracy Test"])
print(
    "KFold validation mean accuracy on test set : {}".format(
        df_val["Accuracy Test"].mean()
    )
)
```

The accuracy on the test dataset shows a comparable result with what we have seen so far.

## 7. Backtesting

We now want to see how our trading strategy would perform on historical data, and this is possible with the help of backtesting. Backtesting allows us to simulate how we would have performed in the past without using any financial capital.

```
In [ ]: # Create column for Strategy Returns by multiplying the daily returns by the pos
# of business the previous day
backtestdata = pd.DataFrame(index=X_validation.index)
validation_prediction = rf_v1.predict(X_validation)
backtestdata["signal_pred"] = validation_prediction
backtestdata["signal_actual"] = y_validation
backtestdata["Market Returns"] = X_validation["Close"].pct_change()
backtestdata["Actual Returns"] = backtestdata["Market Returns"] * backtestdata[
    "signal_actual"
].shift(1)
backtestdata["Strategy Returns"] = backtestdata["Market Returns"] * backtestdata[
    "signal_pred"
].shift(1)
# backtestdata=backtestdata.reset_index()
backtestdata.head()

fig, axs = plt.subplots(2, figsize=(15, 15))
fig.suptitle("Vertically stacked subplots")
axs[0].plot(backtestdata["Strategy Returns"])
axs[0].set_title("Strategy Returns")
axs[1].plot(backtestdata["Actual Returns"])
axs[1].set_title("Bitcoin Daily Returns")

for ax in axs.flat:
    ax.set(xlabel="Predicted Days", ylabel="Returns")

# `backtestdata[['Strategy Returns', 'Actual Returns']].plot()
# `plt.title`('Cumulative Strategy Returns vs Cumulative Actual Returns')
plt.suptitle(
    "Fig. 10: Strategy and Bitcoin Daily Returns Separate Graphs",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

In the above diagrams, we visualize the strategy and Bitcoin daily returns in two separate figures. We put the above figures on the same figure below.

```
In [ ]: backtestdata[["Strategy Returns", "Actual Returns"]].plot(figsize=(15, 8))
plt.title("Strategy Returns vs BITCOIN DAILY Returns chart 2")
plt.suptitle(
    "Fig. 11: Strategy and Bitcoin Daily Returns",
    fontweight="bold",
    horizontalalignment="right",
)
plt.xlabel("Test Days")
```

```
plt.ylabel("Returns")
plt.show()
```

We can clearly see that the strategy and Bitcoin daily return are identical on most test days with the exception of very few days. We can now plot the cumulative returns for both strategy returns and the Bitcoin returns to compare how we expect to perform in the near future if we applied our trading model.

```
In [ ]: backtestdata[["Strategy Returns", "Actual Returns"]].cumsum().plot(figsize=(15,
plt.xlabel("Test Days")
plt.ylabel("Cumulative Returns")
plt.suptitle(
    "Fig. 12: Cumulative Strategy Returns vs Cumulative BITCOIN Returns",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

## 7. Conclusion

In this lesson, we have learned how to apply Bayesian optimization and grid search on our model to improve performance. We also saw how to use baseline models as a reference to the models we will use to predict the buy or sell signals. We anticipate that more could be done to better improve the model, and we ask the students to explore more.

### References

1. Snoek, Jasper, et al. "Practical Bayesian Optimization of Machine Learning Algorithms." *Advances in Neural Information Processing Systems*, vol. 25, 2012.
2. Shalini, Talwar, Shah Pranav, and Shah Utkarsh. "Picking buy-sell signals: A practitioner's perspective on key technical indicators for selected Indian firms." *Studies in Business & Economics* 14.3 (2019).
3. Bergstra, James, et al. "Algorithms for Hyper-parameter Optimization." *Advances in Neural Information Processing Systems*, vol. 24, 2011.  
<https://proceedings.neurips.cc/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf>